

Bahnreisen-Statistiktool

Protokoll des Programmmentwurf zur Prüfung im Fach Advanced Software Engineering

Felix Miller (TINF23B3, 2741471), DHBW Karlsruhe, 03.05.2026

Einführung

Übersicht über die Applikation

Das Bahnreisen-Statistiktool ist ein Werkzeug zur skriptbasierten oder interaktiven Auswertung von eigenen Reisen mit öffentlichen Verkehrsmitteln. Hierzu werden Reisedaten des Check-In-Services „Träwelling“ (<https://traewelling.de/>, <https://github.com/Traewelling/traewelling>) ausgewertet. Über einen Interaktiven Modus kann der Nutzer die Daten Filtern, Gruppieren, Aggregieren und anzeigen, ein Kommandozeilenmodus erlaubt die Nutzung als Skript.

Der Check-In-Service selbst bietet zur Auswertung lediglich sehr eingeschränkte Funktionalitäten. Durch das entwickelte Tool können die Reisedaten deutlich ausführlicher ausgewertet und statistisch analysiert werden.

Starten der Applikation

Um die Applikation zu starten, ist ein C++20-fähiger Compiler erforderlich sowie CMake ab Version 3.17. Unter Windows ist der Build-Prozess lediglich mit MinGW sowie Ninja getestet, dies ist eine Voraussetzung zur Nutzung der automatisierten Build/Test-Pipeline über das compile-Skript.

Um die Applikation zu starten sind folgende Schritte erforderlich:

1. Klonen des Repository
2. compile.bat (Windows) / compile.sh (Linux) ausführen
3. dhw-advanced-software-engineering(.exe) starten. Hierzu sind Parameter erforderlich, die beim Start ohne Parameter erläutert werden. Die notwendigen Testdaten für einen Start sind im Repository der Applikation im Verzeichnis data enthalten.

Sollte der automatische Compile-Prozess fehlschlagen, kann wie folgt vorgegangen werden:

```
cd pfad/zum/repository
```

```
mkdir build
```

```
cd build
```

Konfiguration:

```
cmake .. / cmake -G Ninja
```

Kompilieren:

```
cmake --build .
```

Tests ausführen:

ctest -v

Technischer Überblick

Programmiersprache: C++20

C++ unterstützt sowohl hardwarenahe, performanceorientierte Programmierung als auch moderne, objektorientierte Features sowie funktionale Konzepte. Insbesondere durch die Weiterentwicklungen von modernem C++ seit C++11 sind flexible Abstraktionen bei minimalem Laufzeit-Overhead möglich.

Build-System: CMake

Im Gegensatz zu den ebenfalls geläufigen Makefiles ist CMake unabhängig von verwendetem Betriebssystem und reduziert den Aufwand für eine Portierung auf andere Systeme erheblich. Weiter unterstützt modernes CMake die einfache Integration externer Libraries (hier im Projekt nicht erforderlich, da alles im Repository vorhanden) und überwacht die Compile-Abhängigkeiten.

Softwarearchitektur

Clean Architecture

Domain Code

Distance, Duration, Speed, Station, Coordinates, TransportType, Operator und Count gehören zu den elementaren Datenobjekte der Domäne, die extrahiert werden können und in ähnlichen Projekten wiederverwendet werden können.

Mit Trip und den abgeleiteten Klassen TrainTrip und BusTrip wird die Logik einer Fahrt aus den elementaren Typen der Domäne gebildet.

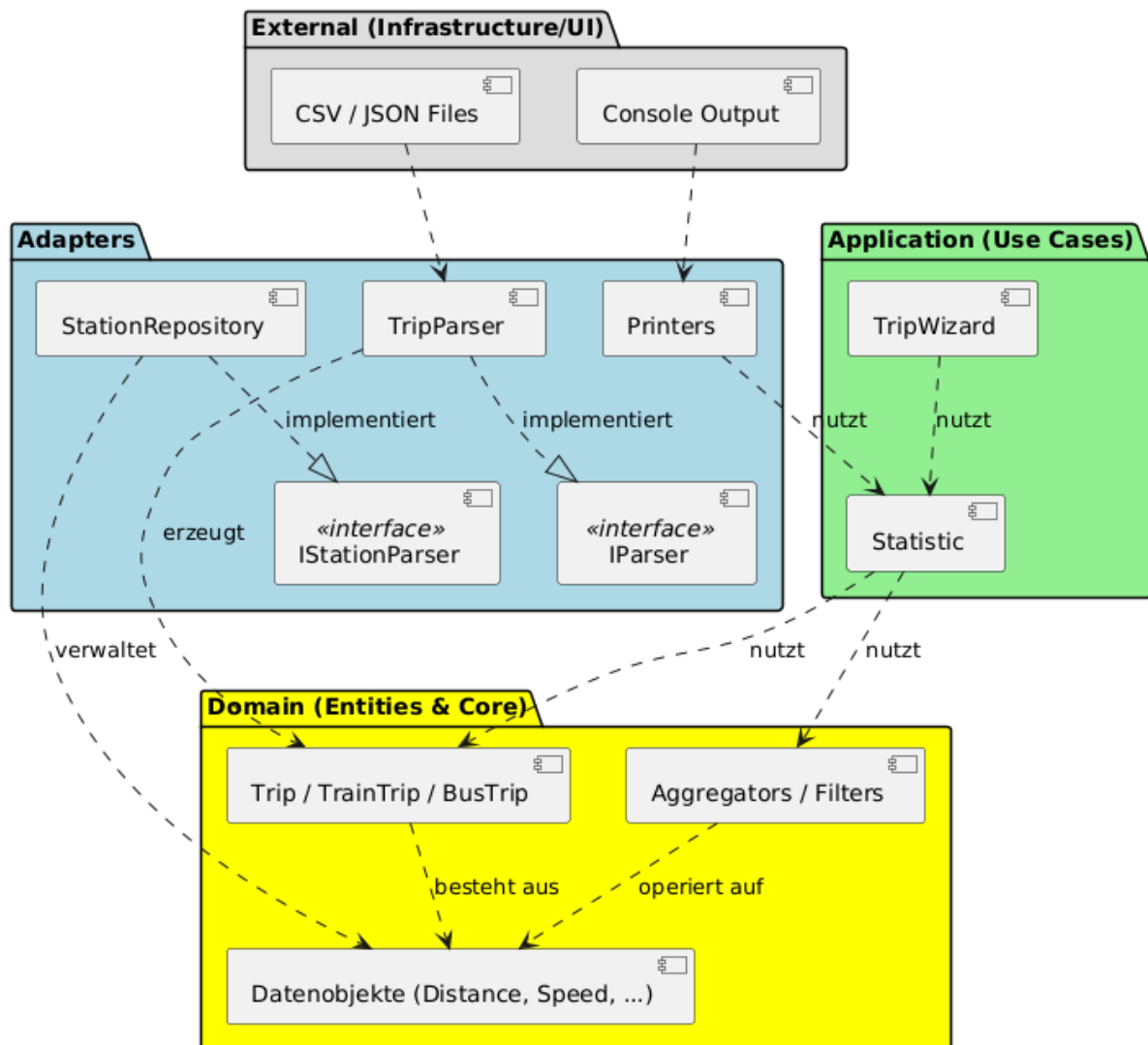
Mathematische Konzepte wie Gruppierung (UniversalGrouper, TripGroupers), Aggregatoren (IAggregator, CountAggregator, DistanceAggregator, DurationAggregator und SpeedAggregator) und Filter (UniversalFilter, Filter) werden ebenfalls in Abhängigkeit zu den elementaren werthaltigen Typen implementiert. Sie können auch anderweitig weiterverwendet werden.

Application Code

Hier gehört die Auswertung der Statistiken (Klasse Statistic) als Anwendungsfall der Applikation zugeordnet. Auch die Klasse TripWizard als interaktives Element zur Datenauswertung ist Teil des Use Cases und damit Teil des Application Codes.

Adapters

Schnittstellen zur Außenwelt finden sich im Namespace Printers (zur Ausgabe von Statistic-Objekten), zum Einlesen der Trips (IParser, TripParser, FakeTripParser, FolderTripParser, ParserFactory) sowie zum Einlesen der Haltestellendaten (IStationParser, ZHVStationParser, FakeStationParser und das StationRepository). Diese Schnittstellen dienen zur Formatkonvertierung von externen Datenquellen zu internen Darstellungsformaten oder zur Darstellung der internen Datenformate zur weiteren Verarbeitung.



Domain Code

Domain Code enthält insbesondere die Entitäts der Applikation und stellt damit den Kern der Domanensprache/Fachsprache dar. Sie ist unabhängig von Details wie Anzeige oder Speicherung und kann so flexibel auch in anderen Anwendungen eingesetzt werden.

Ein Beispiel aus dem Code ist die Klasse `Trip`, diese kapselt die Geschäftslogik und Daten der Applikation und beschreibt das (abstrakte) Konzept einer Fahrt mit einem öffentlichen Verkehrsmittel.

```

#pragma once

#include <string>
#include <vector>
#include <memory>
#include <chrono>

#include "valueObjects/station.h"
#include "valueObjects/duration.h"
#include "valueObjects/distance.h"
#include "valueObjects/transportType.h"

class Trip
{
public:
    Trip(int id, const Station& origin, const Station& destination, int distance, int
duration, const std::chrono::system_clock::time_point startTime, const
TransportType &transportType);
    virtual ~Trip() = default;

    int getId() const;
    Station getOrigin() const;
    Station getDestination() const;
    Distance getDistance() const;
    Duration getDuration() const;
    std::chrono::system_clock::time_point getStartTime() const;

    virtual std::string getTransportName() const = 0;
    TransportType getTransportType() const;

protected:
    int mId;
    Station mOrigin;
    Station mDestination;
    Distance mDistance;
    Duration mDuration;
    std::chrono::system_clock::time_point mStartTime;
    TransportType mTransportType;
};

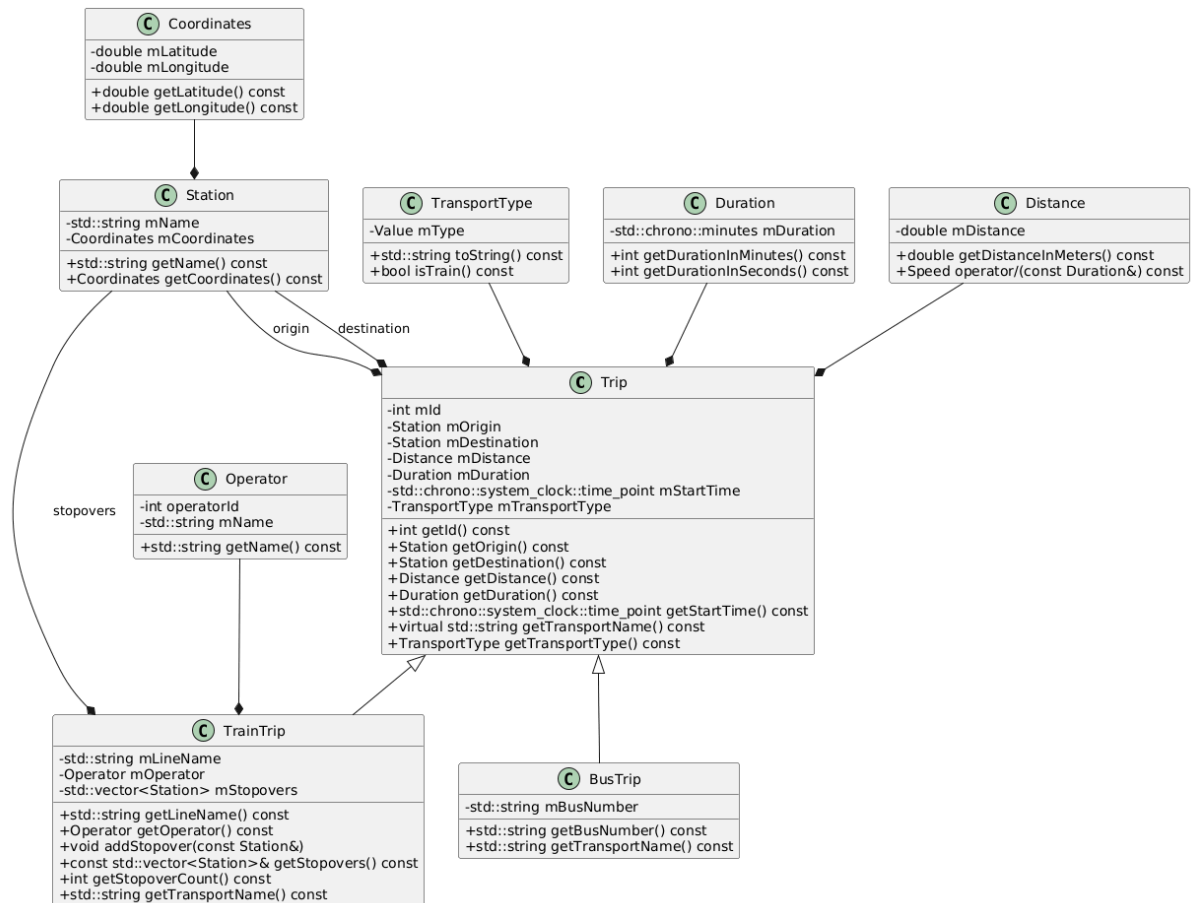
using Trips = std::vector<std::shared_ptr<Trip>>;

```

Dependency Rule

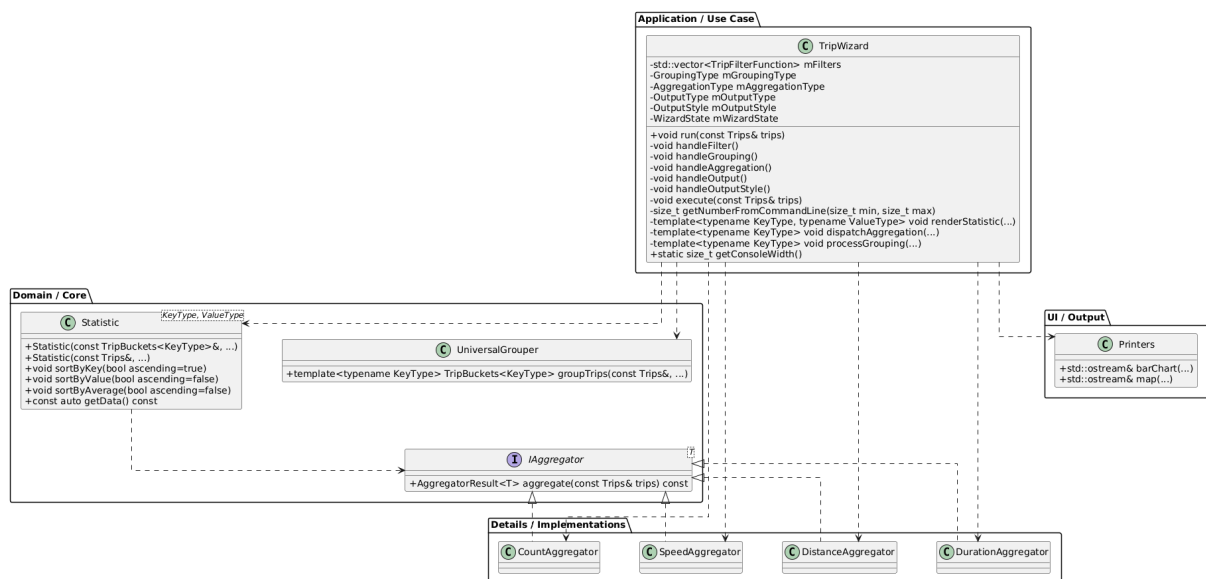
Positiv-Beispiel: Dependency Rule

Die Klasse Trip erfüllt die Dependency Rule. Sie hängt lediglich von den niedrigeren Modulen aus dem Domain Code wie Station, Duration und Speed ab, nicht aber von höherwertigen Modulen wie beispielsweise den Aggregatoren.



Negativ-Beispiel: Dependency Rule

Ein Negativbeispiel für die Dependency Rule ist der TripWizard. Auch wenn er nicht direkt von höheren Schichten abhängt, sollte er in der Anwendungsschicht bzw. Use-Case-Klasse idealerweise nur von Abstraktionen und nicht von konkreten Details abhängen.



SOLID

Analyse SRP

Positiv-Beispiel

```
#pragma once

#include "aggregators/iaggregator.h"

#include "valueObjects/duration.h"

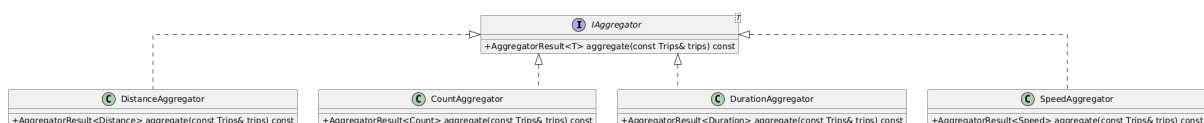
class DistanceAggregator : public IAggregator<Distance>
{
public:
    AggregatorResult<Distance> aggregate(const Trips& trips) const override;
};

#include "aggregators/distanceAggregator.h"

#include "valueObjects/distance.h"

AggregatorResult<Distance> DistanceAggregator::aggregate(const Trips& trips) const
{
    Distance totalDistance = Distance::fromMeters(0);
    for (const auto& trip : trips)
    {
        totalDistance += trip->getDistance();
    }
    return {Distance(totalDistance), static_cast<int>(trips.size())};
}
```

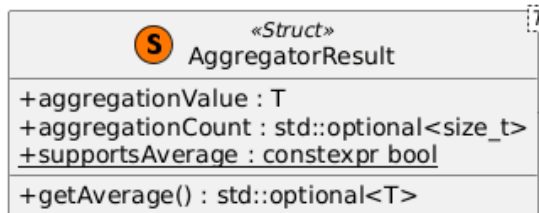
Ein Positiv-Beispiel für das SRP ist die Klasse DistanceAggregator. Sie besitzt die einzige Aufgabe, die Gesamtdistanz für eine Liste von Trips zu berechnen. Andere Aggregationen werden in anderen Klassen implementiert.



Negativ-Beispiel

Ein Negativ-Beispiel ist die Klasse/Struktur (in C++ sind Klassen und Strukturen das gleiche, lediglich die standardmäßige Sichtbarkeit der Member ist unterschiedlich) AggregatorResult aus iaggregator.h. Bei AggregatorResult handelt es sich um einen Containertyp, der sowohl für zählbare Aggregationen (z.B. 20 Kilometer in 10 Fahrten) und nicht zählbare Aggregationen (10 Fahrten in 10 Fahrten ergeben semantisch keinen Sinn) verwendet wird. Für zählbare Aggregationen wird zusätzliche Logik implementiert, den Durchschnittswert der Aggregationen auszurechnen. Hier wird die Speicherung eines Aggregationsergebnis (unabhängig der

Semantik) mit der Berechnung von Durchschnitten vermischt (abhängig von der Semantik), wodurch das Single Responsibility Principle verletzt ist.



Verletzung des SRP:

Die Struktur fungiert als reiner Datencontainer, implementiert aber gleichzeitig fachliche Logik (Durchschnittsberechnung inklusive Typ-Prüfung).

```

#pragma once

#include <vector>
#include <memory>
#include <optional>

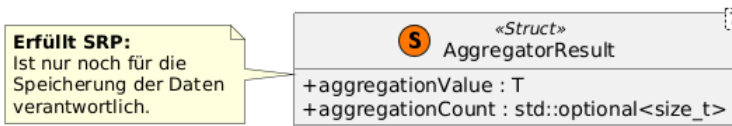
#include "trips/trip.h"

template<typename T>
struct AggregatorResult
{
    T aggregationValue;
    std::optional<size_t> aggregationCount;

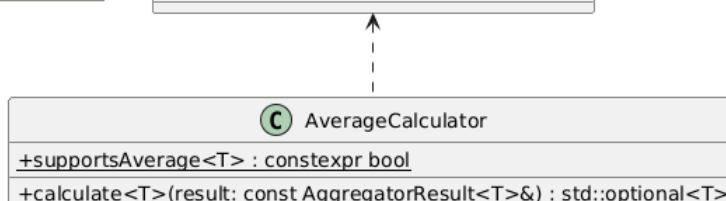
    static constexpr bool supportsAverage = requires(T t, int i) { t / i; };

    std::optional<T> getAverage() const
    {
        if constexpr (supportsAverage)
        {
            if (aggregationCount && *aggregationCount > 0)
            {
                return aggregationValue / *aggregationCount;
            }
        }
        return std::nullopt;
    }
};
    
```

Eine Lösungsmöglichkeit ist es, die Durchschnittsberechnung in eine neue Klasse auszulagern, die sich ausschließlich um die Durchschnittsberechnung kümmert.



Erfüllt SRP:
Ist nur noch für die Speicherung der Daten verantwortlich.



Erfüllt SRP:

Kapselt ausschließlich die mathematische Logik der Auswertung.

Analyse OCP

Positiv-Beispiel

```
#pragma once

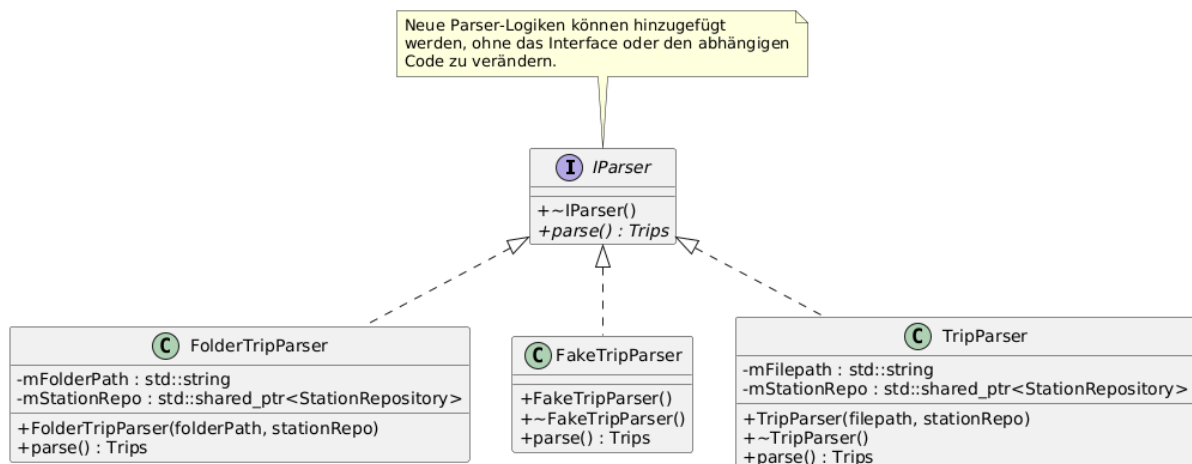
#include <vector>
#include <memory>

#include "trips/trip.h"

class IParser
{
public:
    virtual ~IParser() = default;

    virtual Trips parse() = 0;
};
```

Ein Positivbeispiel für das Open-Closed-Prinzip ist die Klasse IParser. Sie agiert als Interface für verschiedene Parser-Typen und abstrahiert damit von der Implementierung oder der Art der verwendeten Parser. Neue Parser können somit einfach als vom Interface abgeleitete Klasse implementiert werden (Offen für Erweiterung), müssen aber die Klasse IParser dazu nicht verändern (Geschlossen für Modifikationen). Das OCP wurde hier angewendet, um unterschiedliche Arten von Parsern implementieren zu können, die alle die gleiche Schnittstelle nutzen. Beispielsweise existiert ein reiner Parser für eine Datei, es existiert aber auch ein Parser, um mehrere Dateien in einem Ordner zu parsen. Sinnvoll ist diese Struktur auch, wenn später weitere Datenquellen wie APIs erschlossen werden sollen.



Negativ-Beispiel

```
#pragma once

#include "memory"

#include "iparser.h"

class ParserFactory
{
public:
    static std::unique_ptr<IParser> createParserFromFile(const std::string& filePath);
    static std::unique_ptr<IParser> createParserFromFolder(const std::string&
        folderPath);

    static std::unique_ptr<IParser> createMockParser();
};

#include "parser/parserFactory.h"

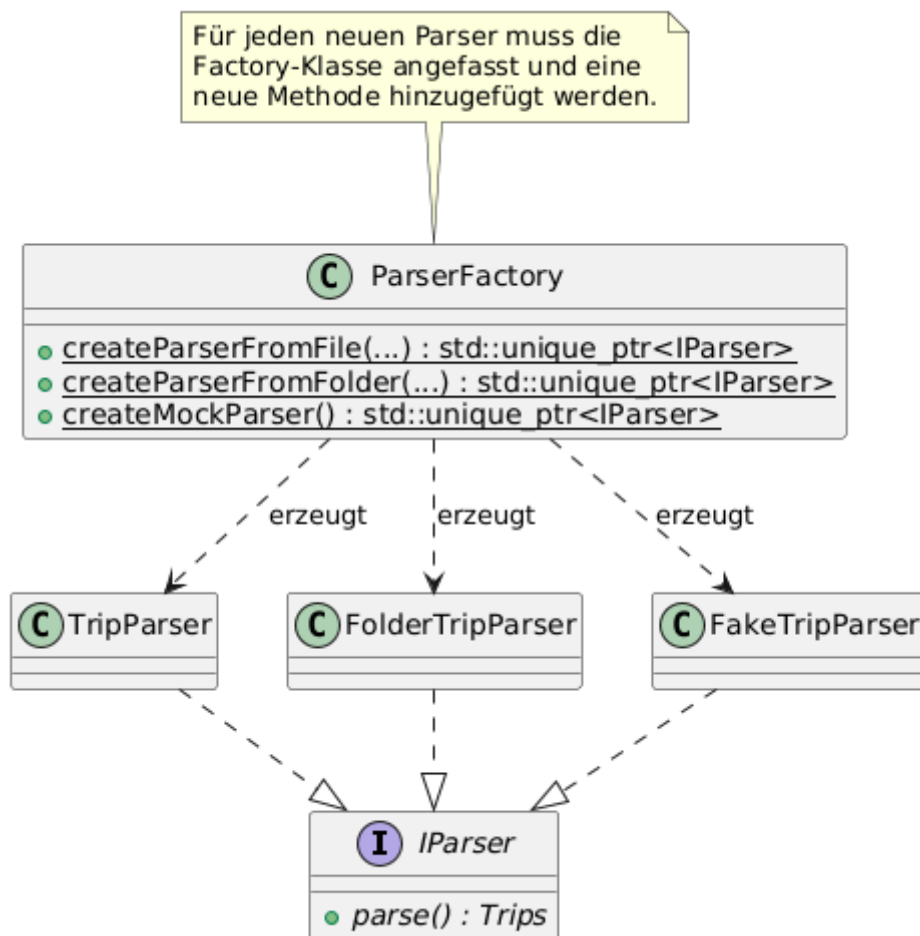
#include "parser/tripparser.h"
#include "parser/faketripparser.h"
#include "parser/folderTripParser.h"

std::unique_ptr<IParser> ParserFactory::createParserFromFile(const std::string &filePath)
{
    return std::make_unique<TripParser>(filePath);
}

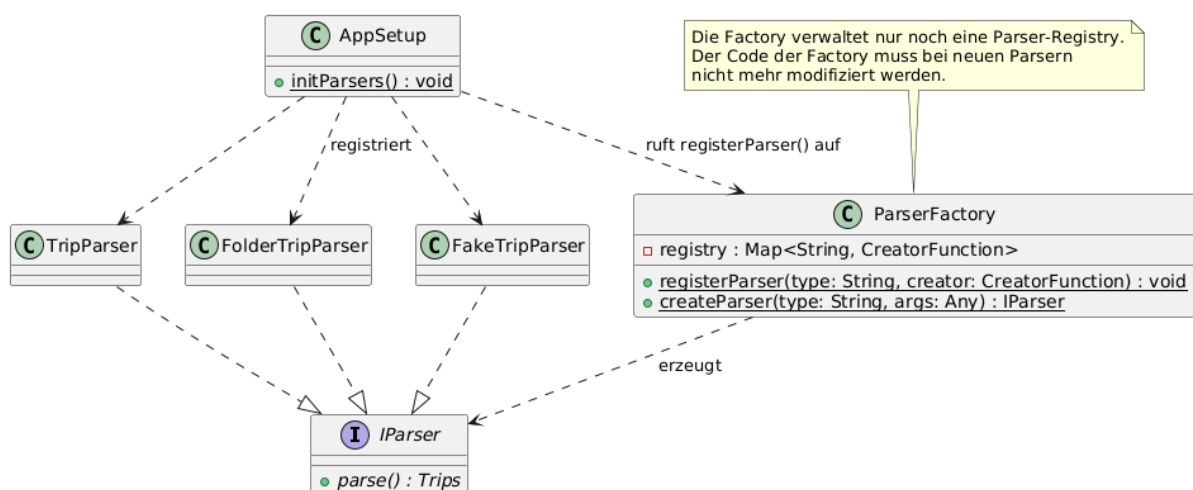
std::unique_ptr<IParser> ParserFactory::createParserFromFolder(const std::string &
    folderPath)
{
    return std::make_unique<FolderTripParser>(folderPath);
}

std::unique_ptr<IParser> ParserFactory::createMockParser()
{
    return std::make_unique<FakeTripParser>();
}
```

Ein Negativbeispiel ist wiederum die Klasse ParserFactory, welche konkrete Instanzen des IParser-Interfaces erzeugt. Hier liegt in der aktuellen Implementierung eine Verletzung des OCP vor. Soll ein neuer Parser integriert werden, muss dieser in der Klasse ParserFactory eingefügt werden und eine Funktion zum Erzeugen des neuen Parsers implementiert werden. Damit ist die Klasse nicht mehr verschlossen gegenüber Modifikationen.



Eine Lösung wäre es, in der ParserFactory eine Registrierungsfunktion anzubieten, bei der sich alle Parser, die Teil der ParserFactory sein sollen, zum Programmstart (oder ggf. zur Kompilierungszeit) registrieren können. Hierdurch können neue Parser hinzugefügt werden, ohne die ParserFactory bearbeiten zu müssen.



Analyse DIP

Positiv-Beispiel

```
#pragma once

#include <vector>
#include <memory>
#include <map>
#include <functional>

#include "trips/trip.h"

template<typename KeyType>
using TripBuckets = std::map<KeyType, Trips>;

class UniversalGrouper
{
public:
    template<typename KeyType>
    static TripBuckets<KeyType> groupTrips(const Trips& trips, const
        std::function<KeyType(const Trip&)>& keyExtractor) ...

    template<typename KeyType>
    static TripBuckets<KeyType> groupTrips(const Trips& trips, const
        std::function<std::vector<KeyType>(const Trip&)>& keysExtractor) ...
};

#pragma once

#include <functional>

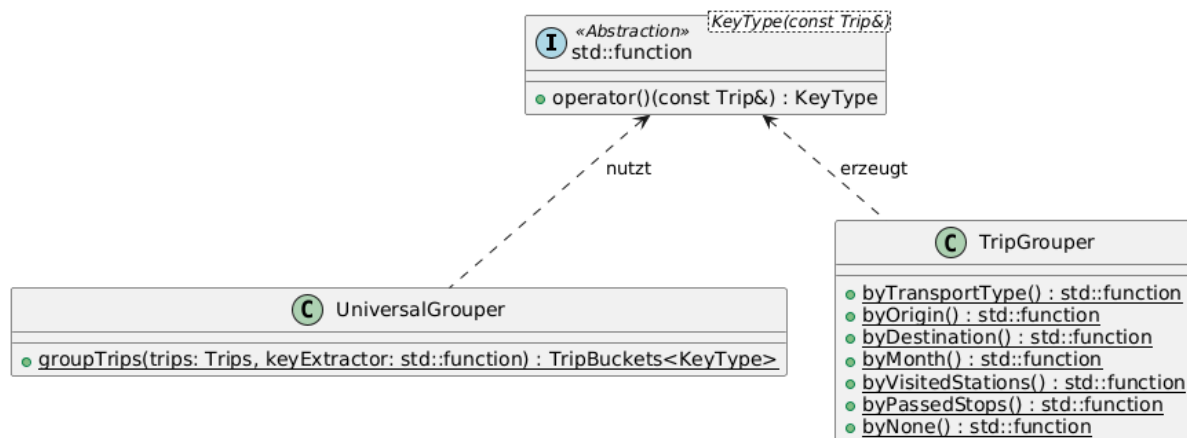
#include "trips/trip.h"
#include "valueObjects/transportType.h"

class TripGrouper
{
public:
    static std::function<TransportType(const Trip&)> byTransportType();
    static std::function<Station(const Trip&)> byOrigin();
    static std::function<Station(const Trip&)> byDestination();
    static std::function<std::string(const Trip&)> byMonth();

    static std::function<std::vector<Station>(const Trip&)> byVisitedStations();
    static std::function<std::vector<Station>(const Trip&)> byPassedStops();

    static std::function<std::string(const Trip&)> byNone();
};
```

Ein Positiv-Beispiel für das Dependency Inversion Principle sind die Klassen UniversalGrouper und TripGrouper. Der UniversalGrouper bekommt, im Sinne der funktionalen Programmierung, eine Rechenvorschrift/Funktion übergeben, die auf die Trips angewendet werde. Anschließend gruppiert dieser die Trips nach den Rückgabewerten der Funktion. Er ist allerdings nicht von den konkreten Gruppierungsfunktionen abhängig, wodurch das DIP erfüllt ist. Konkrete Funktionen, welche die Gruppierungszuordnung implementieren, sind in der Klasse TripGrouper implementiert. Sie hängen nicht vom UniversalGrouper ab.

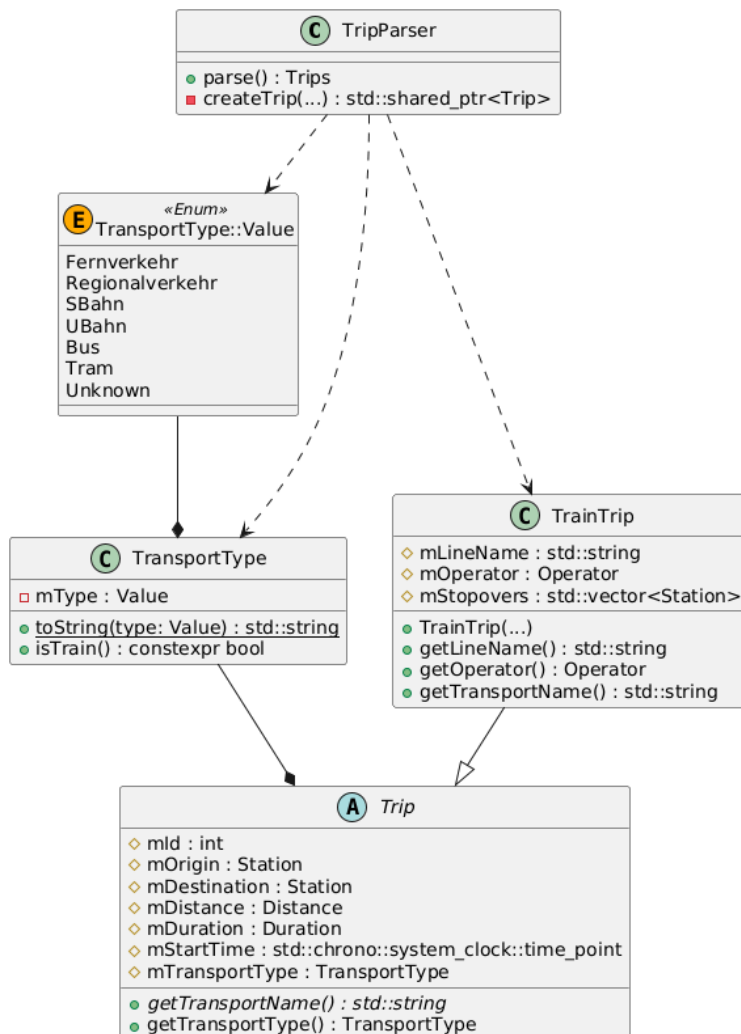


Negativ-Beispiel

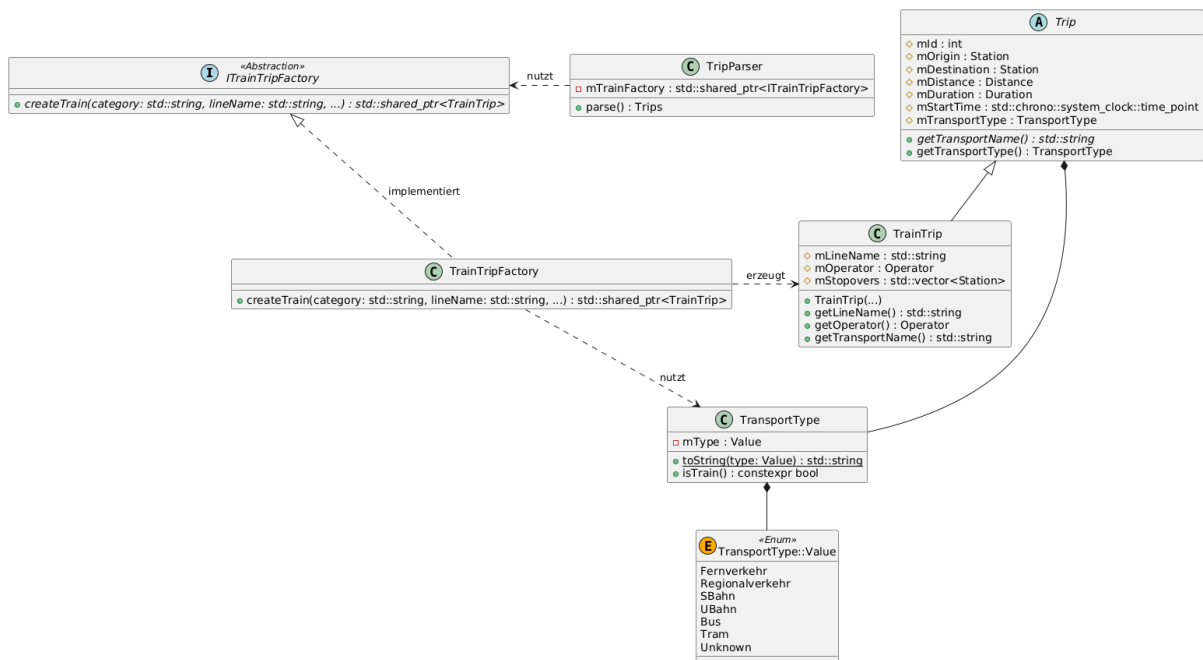
```

std::shared_ptr<TrainTrip> train;
if(category == "express" || category == "regionalExp" || (category ==
"nationalExpress" && lineName.find("IC") != std::string::npos)) //
"regionalExp" is used for FlixTrains, "nationalExpress" is used for ICEs
{
    train = std::make_shared<TrainTrip>(statusId, origin, dest, distance,
    duration, startTime, TransportType::Fernverkehr, lineName, op);
}
else if(category == "regional" || category == "nationalExpress") //
"nationalExpress" is used for french regional trains
{
    train = std::make_shared<TrainTrip>(statusId, origin, dest, distance,
    duration, startTime, TransportType::Regionalverkehr, lineName, op);
}
else if(category == "suburban")
{
    train = std::make_shared<TrainTrip>(statusId, origin, dest, distance,
    duration, startTime, TransportType::SBahn, lineName, op);
}
else if(category == "tram")
{
    train = std::make_shared<TrainTrip>(statusId, origin, dest, distance,
    duration, startTime, TransportType::Tram, lineName, op);
}
else if(category == "subway")
{
    train = std::make_shared<TrainTrip>(statusId, origin, dest, distance,
    duration, startTime, TransportType::UBahn, lineName, op);
}
else
{
    std::cerr << "Unknown category: " << category << '\n';
    continue;
}
  
```

Eine Verletzung des Dependency Inversion Principle ist die Klasse TripParser. Die Erzeugung der verschiedenen Triptypen hängt hierbei direkt von der konkreten Ausgestaltung der Konstruktoren ab. Hierdurch hängt die TripParser-Klasse als äußere Schicht direkt von den Kern-Schichten ab, obwohl sie nach dem DIP von einer Abstraktion abhängen sollte.



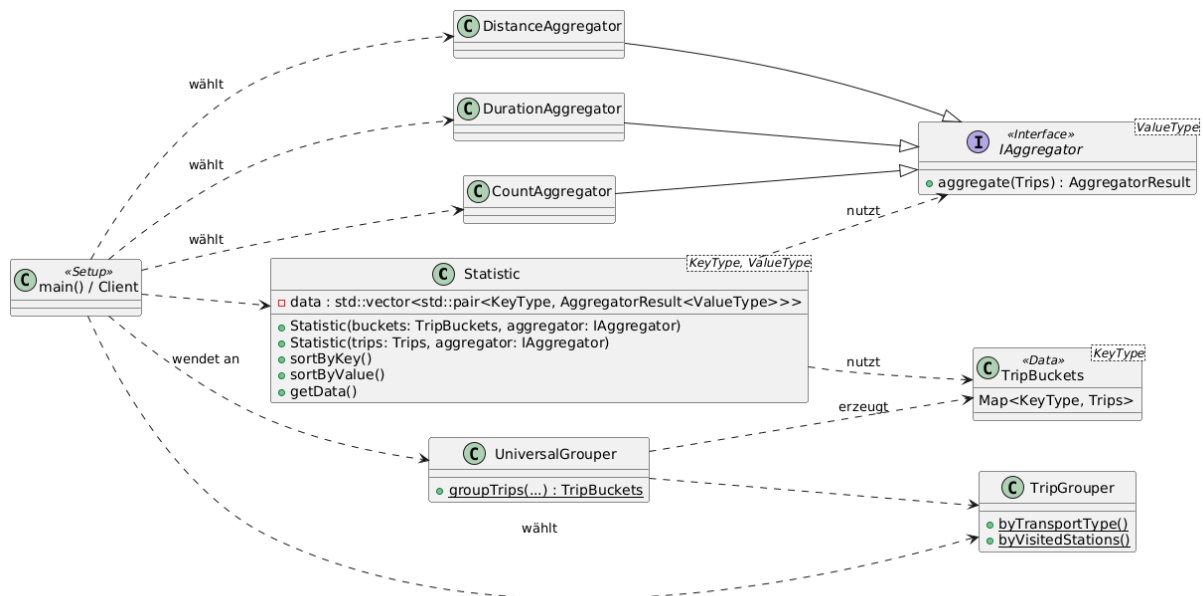
Eine Lösung hierfür wäre eine TrainTrip-Factory, die eine Train-Instanz zurückgibt. Diese abstrahiert die Implementierungsdetails der Transporttypen für den Parser, sodass bei Änderungen im Domänencode oder der Struktur der Eingabedaten lediglich die Abstraktion angepasst werden muss, nicht aber der Parser.



Weitere Prinzipien

Analyse GRASP: Geringe Kopplung

Ein Beispiel für Geringe Kopplung ist die Klasse `Statistic`. Sie ermöglicht es, Trips oder beliebige Gruppierungen von Trips mit einer Aggregatorfunktion zu verknüpfen. Hierzu wird jeweils die Aggregatorfunktion auf die Trips jeder Gruppe angewendet und das Ergebnis im Objekt gespeichert.



```

#pragma once

#include <ostream>

#include "groupers/universalGrouper.h"
#include "aggregators/iaggregator.h"

template<typename KeyType, typename ValueType>
class Statistic
{
public:
    Statistic(const TripBuckets<KeyType> &buckets, const IAggregator<ValueType> &
aggregator) ...

    Statistic(const Trips &trips, const IAggregator<ValueType> &aggregator) ...

    void sortByKey(bool ascending = true) ...

    void sortByValue(bool ascending = false) ...

    void sortByAverage(bool ascending = false) requires
(AggregatorResult<ValueType>::supportsAverage) ...

    friend std::ostream& operator<<(std::ostream &os, const Statistic &statistic) ...

    const auto getData() const { return data; }

private:
    std::vector<std::pair<KeyType, AggregatorResult<ValueType>>> data;
};

template<typename KeyType, typename Agg>
Statistic(const TripBuckets<KeyType>&, const Agg&) -> Statistic<KeyType, decltype
(std::declval<Agg>()).aggregate(std::declval<Trips>()).aggregationValue>;

template<typename Agg>
Statistic(const Trips&, const Agg&) -> Statistic<std::string, decltype(std::declval<Agg>()).
aggregate(std::declval<Trips>()).aggregationValue>;

```

Hierzu hängt sie lediglich von den Interfaces der Aggregatoren sowie des Groupers ab. Der Aufrufer der Klasse übergibt eine beliebige Gruppierung sowie einen beliebigen Aggregator, die in der Statistik-Klasse unabhängig von der Gruppierungs- oder Aggregierungssemantik angewendet werden. Auch der Aufrufer muss durch die Nutzung von Templates und automatischer Typerkennung kein Wissen über die Semantik oder Typen der Gruppierung und des Aggregators besitzen.

Hier liegt eine geringe Kopplung vor, da die Statistik-Klasse ausschließlich von Interfaces abhängt, nicht aber von den konkreten Implementierungen dieser Interfaces. Damit befindet es sich auf der Stufe des polymorphen Aufrufs an ein Interface.

Analyse GRASP: Polymorphismus/Pure Fabrication

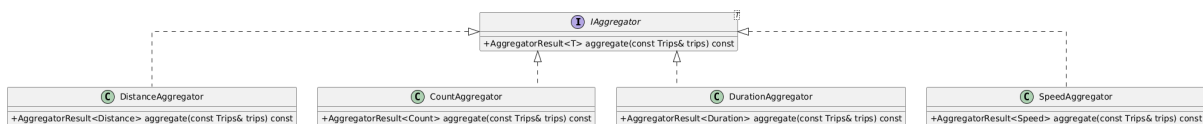
Das Aggregationsinterface IAggregator nutzt Polymorphismus sowohl über Vererbung als auch über Templates. Durch seine rein virtuelle Methode aggregate(const Trips& trips) werden alle vom Interface abgeleiteten Aggregatoren dazu verpflichtet, diese Methode mit einer eigenen Implementierung zu überschreiben. Zusätzlich kann der IAggregator durch die Nutzung von

Templates unterschiedliche Typen verarbeiten, wodurch für neue Typen neue Aggregatoren ohne Anpassungen am Interface erzeugt werden können.

```
template<typename T>
class IAggregator
{
public:
    virtual ~IAggregator() = default;

    virtual AggregatorResult<T> aggregate(const Trips& trips) const = 0;
};
```

Der Polymorphismus wurde hierbei genutzt, um alle Arten von Aggregatoren als Abstraktion des IAggregator-Interface darstellen zu können. Hierdurch ist es in der Statistic-Klasse ausreichend, nur einen Konstruktor für das allgemeine Interface anzugeben. So können Dopplungen in der Code-Logik ohne Konvertierungen verschiedener Aggregatortypen untereinander vermieden werden.



DRY

Im Commit 52db8fdde7fbddfb44fc459a3d4ac58a1faf2074 wurde die rein virtuelle Methode getTransportName() der Klasse TrainTrip durch eine nicht virtuelle Methode ersetzt, wodurch die Implementierung in allen von TrainTrip abgeleiteten Klassen wie beispielsweise TramTrainTrip entfernt werden konnte.

Bei einem Aufruf von getTransportName() wird dem Aufrufer der Verkehrsmitteltyp sowie die Liniennummer als String zurückgegeben. Da die unterschiedlichen abgeleiteten Klassen jeweils anderen schienengebundenen Verkehrsmitteln entsprechen, wird dieser Teil in den abgeleiteten Klassen hard codiert. Da aber ebenso eine rein virtuelle Funktion getTransportType() in TrainTrip existiert, implementieren alle abgeleiteten Klassen nochmals den hard codierten String. Da durch diese Methode bereits eine Methode zur Rückgabe des Verkehrsmitteltyps existiert, kann der hard codierte Verkehrsmitteltypstring in getTransportName() durch den Methodenaufruf von getTransportType() ersetzt werden. Da aber nun alle abgeleiteten Klassen die exakt gleiche Implementierung aufweisen, kann die Implementierung dort entfernt werden und in der TrainTrip-Klasse als nicht virtuelle Funktion implementiert werden.

Vorher	Nachher
<pre>#include "trips/tram_traintrip.h" std::string TramTrainTrip::getTransportName() const { return "Tram (" + mLineName + ")"; } std::string TramTrainTrip::getTransportType() const { return "Tram"; }</pre>	<pre>#include "trips/tram_traintrip.h" std::string TramTrainTrip::getTransportType() const { return "Tram"; } std::string TrainTrip::getTransportName() const { return getTransportType() + " (" + mLineName + ")"; }</pre>

Der String „Tram“ steht als „Magic string“ doppelt im Code. Alle Kindklassen von TrainTrip implementieren beide Funktionen.	Funktion getTransportName() wurde aus allen abgeleiteten Klassen entfernt, die Elternklasse verknüpft stattdessen Transporttyp und Linienname.
---	--

Unit Tests

10 Unit Tests

aggregator_test.cpp

Der Test testet die Funktion der Aggregatoren nach Distanz, Dauer und Geschwindigkeit.

keyType_test.cpp

Der Test testet die als Schlüssel verwendeten Objekte Coordinates, Operator, Station und Transporttype. Es werden jeweils alle Funktionen der Objekte geprüft.

stationRepository_test.cpp

Der Test testet die Funktion des Station Repositories auf Basis der Fake/Mock-Daten. Es werden sowohl gültige Abfragen als auch ungültige Abfragen geprüft.

statistic_test.cpp

Der Test erstellt auf Basis der Fake/Mock-Trips Statistiken zum Transporttyp, zu besuchten sowie zu angefahrenen Stationen. Die Statistik wird anschließend stichprobenmäßig überprüft.

stringUtils_test.cpp

Der Test prüft die Tokenisierung sowie den Ähnlichkeitsvergleich von Strings anhand ausgewählter Beispiele.

transportTypeFilter_test.cpp

Auf die Fake/Mock-Trips wird der Transportfilter angewendet und die Ergebnisse überprüft.

transportTypeGrouper_test.cpp

Die Fake/Mock-Trips werden nach Transporttyp gruppiert, sodass in jeder Gruppe nur ein Eintrag sein sollte. Hierbei wird die konkrete Implementierung der Gruppierung nach Transporttyp getestet, die allgemeine Gruppierungsfunktion wird im folgenden Test getestet.

universalGrouper_test.cpp

Der Test prüft, ob bei nach keinem Kriterium gruppierten Trips weiterhin alle Trips in der gleichen Kategorie vorhanden sind und keine Trips verloren gehen oder dupliziert werden.

valueType_test.cpp

Der Test testet die als Wertobjekte verwendeten Objekte Distance, Duration, Speed und Count inklusive der Operatorüberladung ihrer Rechenoperationen sowie der formatierten Ausgabe.

yearFilter_test.cpp

Es wird getestet, dass sich die Trips nach Jahren filtern lassen. Weiter wird geprüft, dass bei der mehrfachen Anwendung exklusiver Filter keine Trips mehr übrig bleiben.

ATRIIP: Automatic, Thorough und Professional

Automatic:

Die Unit-Tests werden durch die Skripte compile.bat (Windows) bzw. compile.sh (Linux) automatisch ausgeführt. Nur wenn die Unit-Tests fehlerfrei durchlaufen, wird auch die Programmdatei gebaut. So ist sichergestellt, dass der gesamte Build/Test-Prozess inklusive Einrichtung in einem einzigen Schritt geschieht.

Thorough:

Der Buildprozess über CMake aktiviert beim Kompilieren von Debug-Builds für die Tests eine automatische Erfassung der Code Coverage. Diese kann durch das Tool gcovr ausgelesen und ausgegeben werden. Hierbei zeigt sich, dass die zentralen Codebausteine, insbesondere der Domain Code, zu großen Teilen von Tests abgedeckt sind. Nicht abgedeckt sind die Ein- und Ausgabefunktionen, auch die Abdeckung im Application Code ist noch ausbaubedürftig. Hierfür

File	Lines	Exec	Cover
include/parser/folderTripParser.h	1	0	0%
include/parser/tripparser.h	2	0	0%
include/printers/core.h	229	0	0%
include/printers/mapPrinter.h	287	0	0%
include/printers/statisticPrinter.h	456	0	0%
include/printers/tripPrinter.h	14	0	0%
include/tripwizard.h	230	0	0%
include/valueObjects/count.h	3	0	0%
include/valueObjects/duration.h	3	0	0%
include/valueObjects/speed.h	3	0	0%
include/zhvStationParser.h	1	0	0%
src/aggregators/countAggregator.cpp	2	0	0%
src/main.cpp	11	0	0%
src/parser/folderTripParser.cpp	11	0	0%
src/parser/tripparser.cpp	143	0	0%
src/tripwizard.cpp	262	0	0%
src/zhvStationParser.cpp	44	0	0%

müssten weitere Unit-Tests sowie andere Testarten geschrieben werden. Hierbei unterstützt gcovr weiter durch die Anzeige der nicht abgedeckten Zeilen. So können Codestellen mit Testbedarf einfach ermittelt werden.

include/statistic.h	182	23	12%
include/aggregators/iaggregator.h	20	3	15%
src/parser/parserFactory.cpp	6	2	33%
src/trips/bustrip.cpp	6	2	33%
src/trips/traintrip.cpp	15	7	46%
include/trips/bustrip.h	2	1	50%
include/trips/traintrip.h	2	1	50%
include/groupers/universalGrouper.h	29	19	65%
include/valueObjects/distance.h	3	2	66%
src/aggregators/speedAggregator.cpp	11	9	81%
src/groupers/tripGroupers.cpp	40	33	82%
src/iStationParser.cpp	16	14	87%
src/stationRepository.cpp	22	20	90%
src/fakeStationParser.cpp	12	11	91%
src/stringUtils.cpp	34	32	94%
include/iStationParser.h	2	2	100%
include/parser/faketripparser.h	3	3	100%
include/parser/iparser.h	1	1	100%
include/trips/trip.h	1	1	100%
include/valueObjects/coordinates.h	3	3	100%
include/valueObjects/operator.h	1	1	100%
include/valueObjects/station.h	6	6	100%
include/valueObjects/transportType.h	6	6	100%
src/aggregators/distanceAggregator.cpp	5	5	100%
src/aggregators/durationAggregator.cpp	5	5	100%
src/filters/filter.cpp	14	14	100%
src/filters/universalFilter.cpp	4	4	100%
src/parser/faketripparser.cpp	36	36	100%
src/trips/trip.cpp	16	16	100%
src/valueObjects/coordinates.cpp	3	3	100%
src/valueObjects/count.cpp	23	23	100%
src/valueObjects/distance.cpp	37	37	100%
src/valueObjects/duration.cpp	37	37	100%
src/valueObjects/operator.cpp	8	8	100%
src/valueObjects/speed.cpp	15	15	100%
src/valueObjects/station.cpp	7	7	100%
src/valueObjects/transportType.cpp	14	14	100%

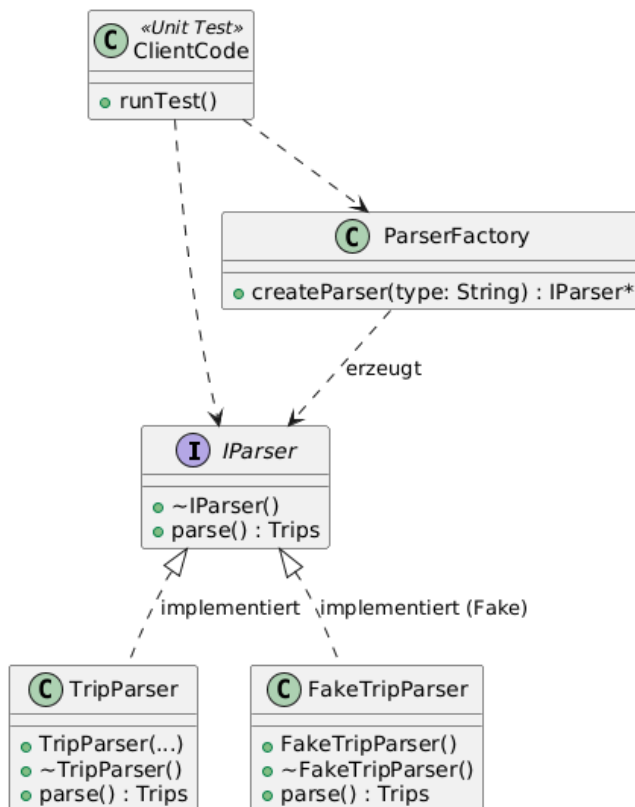
Professional:

Tests auf Basis von C-Assertions via `assert(ist == soll)` erfordern unbedingt, dass Debug-Symbole im Code erhalten bleiben. Hierzu müssen die Tests als Debug-Build gebaut werden. Dies wird durch die Pipeline sichergestellt. Weiter ist CMake als Testorchestrator prinzipiell plattformunabhängig, wodurch auf allen gängigen Entwicklungsplattformen die Unit Tests ausgeführt werden können. Außerdem ist sichergestellt, dass die Testumgebung streng von der Produktionsumgebung getrennt ist.

Fakes und Mocks

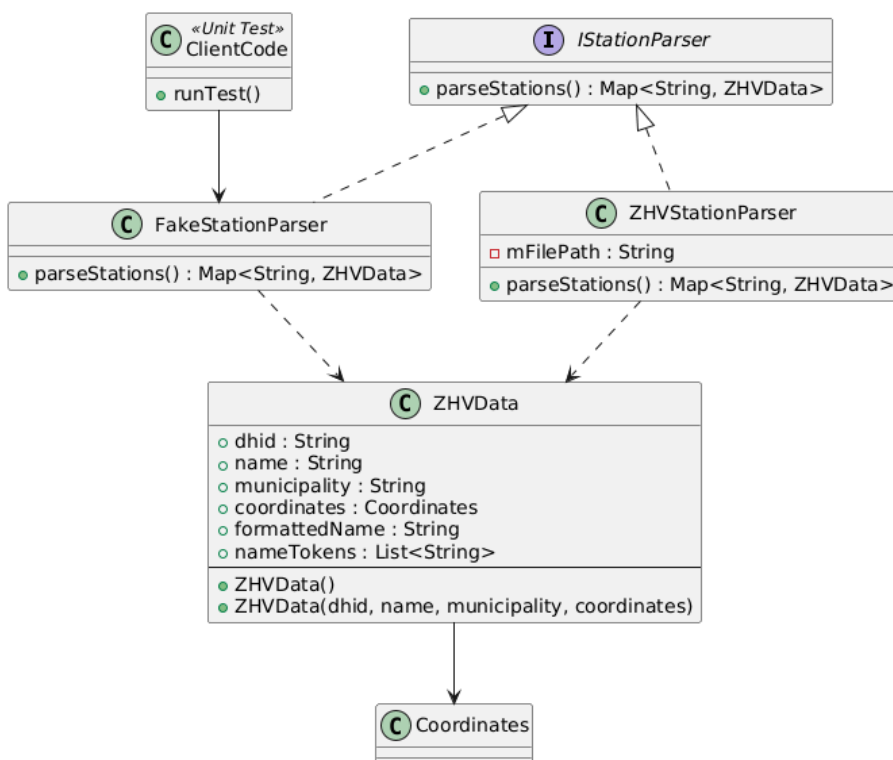
Fake/Mock 1: FakeTripParser

Der Produktiv-TripParser erfordert den Zugriff auf eine Datei mit validen Trip-Daten. Das Öffnen, Parsen und Verarbeiten von potenziell fehlerhaften und nicht persistenten Daten sollte in Unit-Tests vermieden werden. Deswegen eignet sich der TripParser schlecht für den Einsatz in Unit Tests. Aus diesem Grund wird ein FakeTripParser mit einem zur Compile-Zeit feststehenden Datensatz implementiert. Dieser erbt ebenfalls vom Parser-Interface und kann somit direkt anstelle des TripParsers eingesetzt werden. Mithilfe der ParserFactory kann sich bequem eine Instanz des FakeTripParsers zurückgegeben werden.



Fake/Mock 2: FakeStationParser

Eine ähnliche Situation ergibt sich bei den Implementierungen des `IStationParser`-Interfaces. Der `ZHVStationParser` benötigt Zugriff auf die CSV-Datei mit allen Haltestellendaten. Für Unit Tests existiert deswegen die Klasse `FakeStationParser`, die einen vordefinierten Testdatensatz im Speicher bereithält und diesen bei Aufruf zurückgibt.



Domain Driven Design

Ubiquitous Language

Bezeichnung	Bedeutung im fachlichen Kontext	Begründung, dass die Bezeichnung teil der Ubiquitous Language ist
Trip	Die ununterbrochene Fahrt mit einem einzelnen Verkehrsmittel zwischen zwei Stationen.	In der Domäne bezeichnet Trip explizit keine Reisekette mit beispielsweise Umsteigebeziehungen, mehreren Verkehrsmitteln oder Hin- und Rückfahrt, sondern lediglich den Prozess Einsteigen → Fahren → Aussteigen. Es unterscheidet sich damit von der geläufigen Definition einer Reise oder eines Ausflugs
Aggregator	Eine Rechenvorschrift, die eine Liste von Fahrten zu einem Kennwert (Distanz, Dauer, Geschwindigkeit, Anzahl) zusammenfasst.	Aggregator/Aggregation ist hier explizit im Kontext der Statistik und Datenanalyse zu sehen. Aus einer gegebenen Liste werden gleichartige Informationen zusammengefasst. Dies steht im Gegensatz zu anderen domänenspezifischen Definitionen wie beispielsweise Nachrichtenaggregatoren oder Aggregatoren als Entwurfsmuster für Software.
Stopover	Ein Zwischenhalt innerhalb eines Trips, der weder Start noch Ziel ist.	Innerhalb der Domäne muss semantisch zwischen betretenen Halten (Origin/Destination) und besuchten, aber nicht betretenen Halten (Stopover) unterschieden werden.
Line name	Die öffentlich bekannte Bezeichnung einer Verbindung (z. B. "RE1", "S3").	Wichtig zu wissen ist, dass eine Gruppierung nach Liniennamen semantisch nicht sinnvoll ist, da ein Liniename in der Domäne

		kein Primärschlüssel für eine tatsächliche Linie ist. So existieren in Baden-Württemberg mehrere S-Bahnen mit dem Liniennamen S1, ebenso existieren deutschlandweit mehrere Linien mit der Bezeichnung RE1.
--	--	---

Repositories

```
#pragma once

#include <unordered_map>
#include <memory>
#include <string>
#include <vector>
#include <optional>

#include "iStationParser.h"
#include "valueObjects/station.h"

class StationRepository
{
public:
    StationRepository(const IStationParser &stationParser);

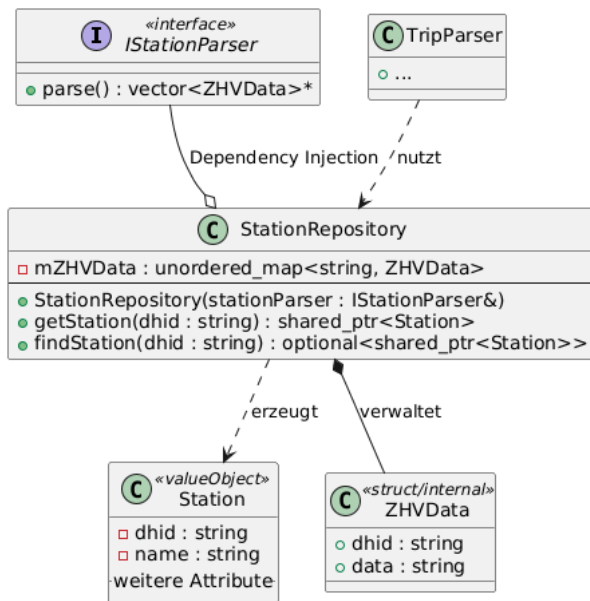
    std::shared_ptr<Station> getStation(const std::string &dhid) const;
    std::optional<std::shared_ptr<Station>> findStation(const std::string &dhid) const;

private:
    std::unordered_map<std::string, ZHVDData> mZHVDData;

    friend class TripParser;
};
```

Das StationRepository agiert als Vermittler zwischen der Persistenzschicht (zHV-csv-„Datenbank“) und der Geschäftslogik. Der Zugriff auf die Haltestellendaten in Form von Stationsobjekten wird über den Zugriff mit dem Primärschlüssel (DHIDs) abstrahiert. Technische Details wie die Normalisierung der DHIDs werden so vor dem Aufrufer verborgen.

Das Repository wird über Dependency Injection (IStationParser) von der Datenquelle entkoppelt, so kann das Repository auch mit Fake/Mock-Daten getestet werden. Weiter bietet das Repository Vorteile, da bei einer Änderung an der Datenquelle lediglich dort Änderungen, nicht aber an allen Zugriffsstellen ausgeführt werden müssen.



Aggregates

```

#pragma once

#include "trip.h"

#include <vector>
#include <string>

#include "valueObjects/operator.h"

class TrainTrip : public Trip
{
public:
    TrainTrip(int id, const Station& origin, const Station& destination, int
distanceKm, int duration, const std::chrono::system_clock::time_point startTime,
const TransportType& transportType, const std::string& lineName, const Operator&
op);
    ~TrainTrip() override = default;

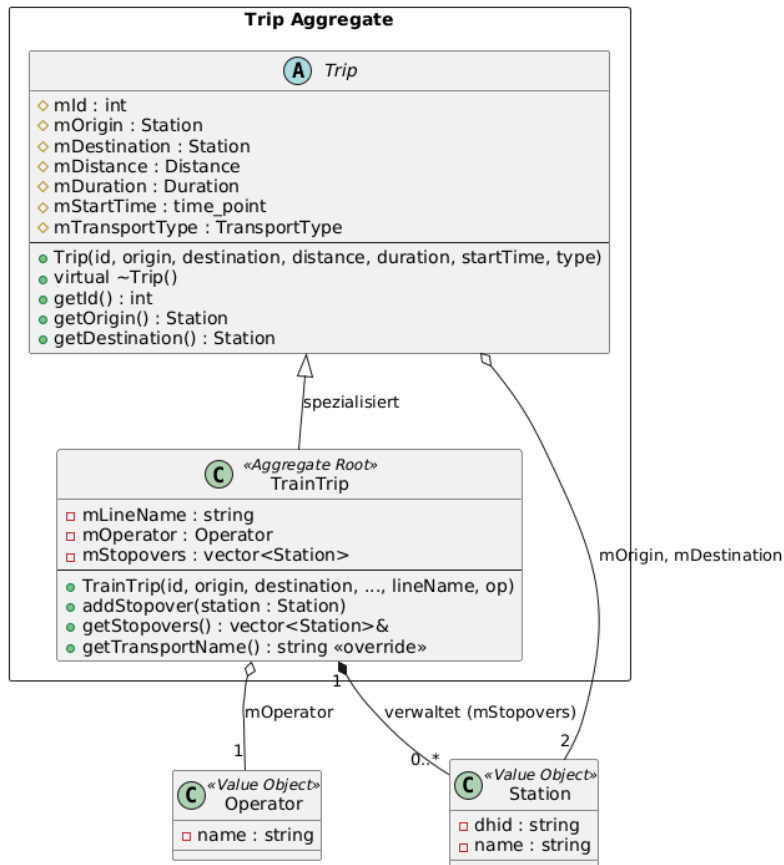
    std::string getLineName() const;
    Operator getOperator() const;

    void addStopover(const Station& station);
    const std::vector<Station>& getStopovers() const;
    int getStopoverCount() const;

    std::string getTransportName() const override;

protected:
    std::string mLineName;
    Operator mOperator;
    std::vector<Station> mStopovers;
};
  
```

Das Aggregat TrainTrip bündelt Eigenschaften wie Start, Ziel und Zwischenhalte in eine gemeinsame Einheit mit einer eindeutigen ID, die den TrainTrip identifiziert. So gibt es einen eindeutigen Zugriffspunkt für die Daten einer Zugfahrt und die Daten müssen nicht aus verteilten Objekten oder (In-Memory-)Datenbanken zusammengefügt werden. Da die einzige Veränderungsmöglichkeit das Hinzufügen von Zwischenhalten ist, muss der TrainTrip von Anfang an korrekt initialisiert sein, wodurch die Vollständigkeit der relevanten Daten sichergestellt ist.



Entities

```
#pragma once

#include <string>

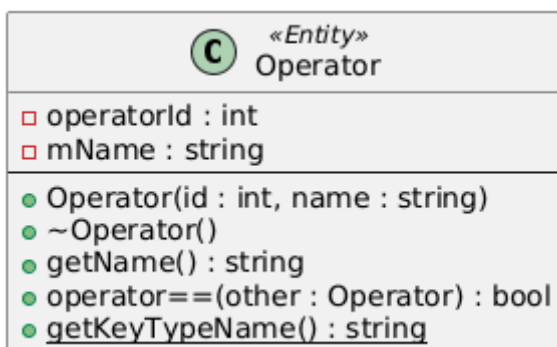
class Operator
{
public:
    Operator(int id, const std::string& name);
    ~Operator() = default;

    std::string getName() const;

    bool operator==(const Operator& other) const;

    static std::string getKeyTypeName();
private:
    int operatorId;
    std::string mName;
};
```

Bei der Klasse Operator handelt es sich um ein Entity. Unabhängig vom Namen des Entities sind zwei Operatoren nur dann gleich, wenn sie die gleiche ID haben. Selbst wenn zwei Betreiber ähnliche oder ggf. gleiche Namen führen (z.B. Arverio Baden-Württemberg oder Arverio Bayern, beide gelegentlich einfach nur Arverio bezeichnet), handelt es sich um unterschiedliche Unternehmen. Auch durch eine Namensänderung eines Betreibers (wie beispielsweise von Go-Ahead zu Arverio) ändert nichts daran, dass es sich um das gleiche Unternehmen handelt.



Value Objects

```
#pragma once

#include <ostream>

class Coordinates
{
public:
    Coordinates(double latitude, double longitude) : mLatitude(latitude), mLongitude(longitude) {}

    double getLatitude() const { return mLatitude; }
    double getLongitude() const { return mLongitude; }

    friend std::ostream& operator<<(std::ostream& os, const Coordinates& coords);

private:
    double mLatitude;
    double mLongitude;
};
```

Die Klasse Coordinates ist ein Value Object. Sie fasst Breiten- und Längengrade eines geografischen Punktes (z.B. einer Haltestelle) zu einer logischen, unveränderlichen Einheit zusammen. Die Unveränderlichkeit wird durch die privaten Membervariablen sowie den Konstruktor garantiert. Coordinates wird zudem lediglich durch seine Werte beschrieben und besitzt somit keine Identität. Da Koordinaten nicht als primitiver Datentyp existieren, kann durch das Value Object eine strukturierte Darstellung (inklusive semantischer Kontrolle, was bei primitiven Datentypen wie double nicht möglich wäre) von Punkten erreicht werden.

 «Value Object» Coordinates
❑ mLatitude : double ❑ mLongitude : double
● Coordinates(latitude : double, longitude : double) ● getLatitude() : double ● getLongitude() : double
● «friend» operator<<(os : ostream, coords : Coordinates) : ostream&

Refactoring

Code Smells

Code Smell 1: Long Method

```
std::unordered_map<std::string, ZHVDData> ZHVStationParser::parseStations() const
{
    std::unordered_map<std::string, ZHVDData> stationDataMap;

    std::ifstream file(mFilePath);
    if (!file.is_open())
    {
        throw std::runtime_error("Could not open file: " + mFilePath);
    }

    std::string line;

    while (std::getline(file, line))
    {
        if(line.empty()) continue;

        auto tokens = parseCSVLine(line);

        // expect 5 tokens (DHID, Name, Lat, Lon, Municipality)
        if(tokens.size() >= 5)
        {
            std::string dhid = tokens[0];
            std::string name = tokens[1];
            std::string latStr = tokens[2];
            std::string lonStr = tokens[3];
            std::string municipality = tokens[4];

            // replace comma with dot for conversion to double
            std::replace(latStr.begin(), latStr.end(), ',', '.');
            std::replace(lonStr.begin(), lonStr.end(), ',', '.');

            try
            {
                double latitude = std::stod(latStr);
                double longitude = std::stod(lonStr);

                ZHVDData data(dhid, name, municipality, Coordinates(latitude, longitude));
                stationDataMap[dhid] = data;
            }
            catch (const std::exception& e)
            {
                std::cerr << "Skipping invalid coords for " << dhid << "\n";
                continue;
            }
        }
    }

    return stationDataMap;
}
```

Bei der Methode `ZHVStationParser::parseStations()` handelt es sich um eine „Long Method“. Eine Lösung hierfür wäre es, Funktionalitäten in eigene Methode auszulagern, siehe Pseudocode. Hierdurch sinkt die Methodenlänge erheblich, der Code ist sprechender und selbstdokumentierend.

```
std::unordered_map<std::string, ZHVData> ZHVStationParser::parseStations() const
{
    std::unordered_map<std::string, ZHVData> stationDataMap;

    std::vector<std::string> lines = readCSVFile(stationDataFilePath);

    for (const auto& line : lines)
    {
        auto tokens = parseCSVLine(line);

        // expect 5 tokens (DHID, Name, Lat, Lon, Municipality)
        if(tokens.size() >= 5)
        {
            auto &[dhid, name, latStr, lonStr, municipality] = parseTokens(tokens);

            auto coordinates = parseCoordinates(latStr, lonStr);

            try
            {
                ZHVData data(dhid, name, municipality, coordinates);
                stationDataMap[dhid] = data;
            }
            catch (const std::exception& e)
            {
                std::cerr << "Skipping invalid coords for " << dhid << "\n";
                continue;
            }
        }
    }

    return stationDataMap;
}
```

Code Smell 2: Long Parameter List

Der Konstruktor der Klasse `TrainTrip` besitzt mit 9 Parametern eine lange Parameterliste. Diese kann durch die Aufteilung in mehrere Parameter-Objekte reduziert werden, wie als Pseudocode dargestellt.

```
TrainTrip(int id, const Station& origin, const Station& destination, int
distanceKm, int duration, const std::chrono::system_clock::time_point startTime,
const TransportType& transportType, const std::string& lineName, const Operator&
op);
```

```

struct BaseTripData
{
    Station origin;
    Station destination;
    int distanceKm;
    int duration;
    std::chrono::system_clock::time_point startTime;
    TransportType transportType;
};

struct TrainTripData : public BaseTripData
{
    std::string lineName;
    Operator op;
};

TrainTrip(int id, const TrainTripData& data));

```

2 Refactorings:

Refactoring 1: Extract to Method

Commits e3ca70965ac57292cb1d999f069357b4c13a7d80 und 2897a388000bf916bf1cf6ddfe483f7edfe886a4.

Vorher:

```

class TripParser : public IParser
{
public:
    explicit TripParser(const std::string& filepath, const
        std::shared_ptr<StationRepository>& stationRepo);
    ~TripParser() override = default;

    Trips parse() override;

private:
    std::string mFilepath;
    std::shared_ptr<StationRepository> mStationRepo;
};

```

Die Klasse TripParser implementiert lediglich die Funktion parse() in ca. 180 LOC.

Nachher:

```

class TripParser : public IParser
{
public:
    explicit TripParser(const std::string& filepath, const
        std::shared_ptr<StationRepository>& stationRepo);
    ~TripParser() override = default;

    Trips parse() override;

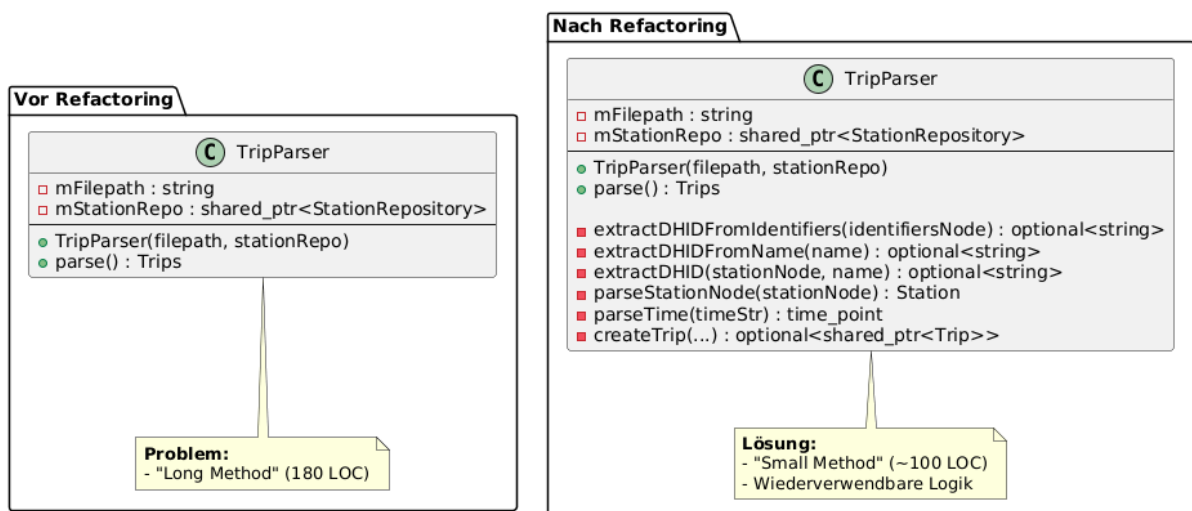
private:
    std::string mFilepath;
    std::shared_ptr<StationRepository> mStationRepo;

    std::optional<std::string> extractDHIDFromIdentifiers(const json &identifiersNode)
        const;
    std::optional<std::string> extractDHIDFromName(const std::string &name) const;
    std::optional<std::string> extractDHID(const json &stationNode, const std::string &
        name) const;

    Station parseStationNode(const json &stationNode) const;
    std::chrono::system_clock::time_point parseTime(const std::string &timeStr) const;
    std::optional<std::shared_ptr<Trip>> createTrip(int statusId, const Station &
        origin, const Station &dest, int distance, int duration, const
        std::chrono::system_clock::time_point &startTime, const std::string &category,
        const std::string &lineName, const Operator &op) const;
};

```

Aus der Funktion parse() wurden die (teilweise mehrfach verwendeten) Funktionen extractDHID() (und daraus die Funktionen extractDHIDFromName()/extractDHIDFromIdentifiers()) sowie parseStationNode(), parseTime() und createTrip() extrahiert. Hierdurch konnte die Anzahl der LOC von 180 auf ca. 100 reduziert werden, woran sich ein weiteres Refactoring anschließen könnte.



Refactoring 2: Rename Method

Commit 7ba90ee70e70bed8f558a96f5be59a679a773270

Vorher:

```

template <typename T>
struct PrettyWrapper
{
    const T &value;
    bool useAverage = false;
    size_t width;

    PrettyWrapper(const T &value, bool useAverage = false, size_t width = 80) : value
    (value), useAverage(useAverage), width(width) {}
};

template <typename T>
inline PrettyWrapper<T> pretty(const T &value, bool useAverage = false, size_t width =
80)
{
    return PrettyWrapper<T>{value, useAverage, width};
}

```

Die Struktur `PrettyWrapper` und die dazugehörige Erzeugungsfunktion `pretty()` erklären durch ihren Namen nicht, was ihre Funktion ist. Aus der Zugehörigkeit zum Namespace lässt sich lediglich erkennen, dass es sich um Wrapper-Funktionen für die Konsolenausgabe handeln muss. Die konkreten Aufgaben der Wrapper-Funktionen und ihr Einsatzzweck werden durch die Methodennamen nicht klar.

Nachher:

```

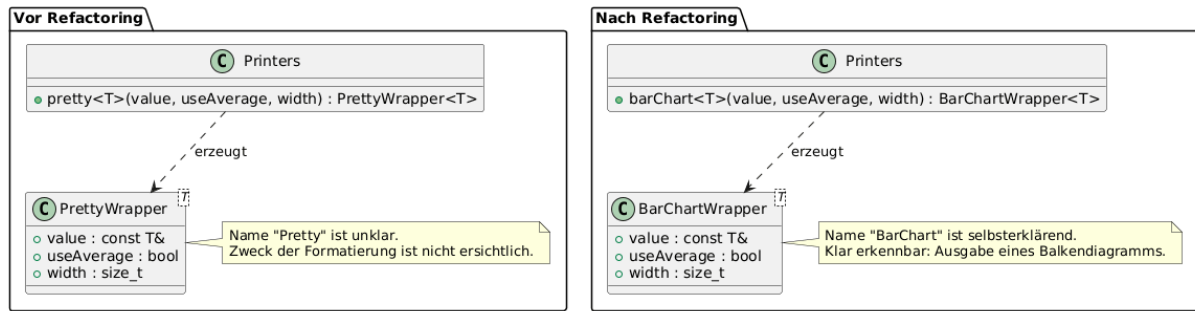
template <typename T>
struct BarChartWrapper
{
    const T &value;
    bool useAverage = false;
    size_t width;

    BarChartWrapper(const T &value, bool useAverage = false, size_t width = 80) : value
    (value), useAverage(useAverage), width(width) {}
};

template <typename T>
inline BarChartWrapper<T> barChart(const T &value, bool useAverage = false, size_t
width = 80)
{
    return BarChartWrapper<T>{value, useAverage, width};
}

```

Aus der Struktur `BarChartWrapper` und aus der Funktion `barChart` wird für den Aufrufer klar, dass es sich um die Wrapper-Methode zur Ausgabe eines Balkendiagramms handeln muss. Hierdurch wird der Code selbstdokumentierend.

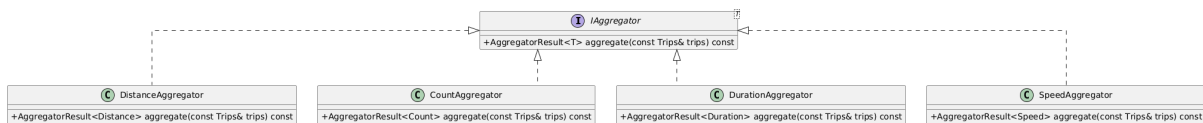


Entwurfsmuster

Entwurfsmuster: Strategy

Zur Implementierung der Logik von Summen- oder Durchschnittsbildung eines Wertes über eine Liste von Trips wurde mit den Aggregator-Klassen das Entwurfsmuster Strategy eingesetzt. So können verschiedene Algorithmen zur Aggregation von Werten austauschbar unter dem gleichen Interface verwendet werden. Diese Logik ist so getrennt von beispielsweise dem TripWizard, der lediglich den verwendeten Aggregator erzeugen muss und an die Statistik übergibt, aber keine Informationen über die Funktionsweise des Aggregators benötigt.

Die Algorithmen können so an verschiedenen Stellen wiederverwendet werden, ausgetauscht oder erweitert werden, ohne etwas an der grundsätzlichen Logik ändern zu müssen. Damit ist das OCP erfüllt.



Entwurfsmuster: Decorator/Wrapper

Die formatierte Konsolenausgabe von Trips, Statistiken und Karten wird über Decorators bereitgestellt. Hierzu werden die anzuzeigenden Daten in einem Wrapper verpackt, für welchen die Operatorüberladung `<<` implementiert ist (Konsolenausgabe). So können die einzelnen Objekte wie Trip und Statistic (abgesehen von einer einfachen Debug-Ausgabe) frei von Anzeigelogik bleiben, wodurch das SRP eingehalten wird. Hiermit bleibt auch die Trennung von Daten und Darstellung gewahrt, da so ggf. das gleiche Datenobjekt mithilfe mehrerer Wrapper/Decorators anders dargestellt werden kann.

